SPRINT 02, QUALIDADE E CI

CI parte 1, validações locais e containers

Antes do push, o laptop é a primeira pipeline. Hooks, Husky, Docker e a imagem que vai para o registry.

Aula	05 de 14
Data	11 de maio de 2026, segunda-feira, 10h00 às 12h00
Professor	José Romualdo da Costa Filho
Módulo	13, Engenharia de Software, Turma T13

Agenda da aula

inteli.

Aula invertida, 2 horas, bem prática. Vocês leram o auto-estudo, aqui a gente roda. Quatro blocos, dois hands-on grandes, fechamento curto.

Bloco	Atividade	Duração
10h00, Daily Time	Check-in da sprint, cronômetro de 15 min	15 min
10h15, Bloco 1	Git hooks, Husky, scanner de secrets, hands-on A	30 min
10h45, Bloco 2	Virtualização, Docker, Dockerfile, hands-on B	45 min
11h30, Bloco 3	Registry, vulnerabilidades, demo com Trivy	20 min
11h50, Fechamento	Quiz, checklist antes do push, auto-estudo	10 min

DAILY, 15 MIN

Daily Time

Como está a sprint? O que já está pronto do que ficou combinado na Aula 04, MR aberta, revisão feita, triagem do Sonar? Quem ficou travado diz em uma frase. Uma volta rápida, sem rodeio.

15:00

INICIAR PAUSAR ZERAR

Objetivos de aprendizagem

inteli.

No fim da aula, cada dupla sai com três coisas funcionando no próprio repositório: um hook que barra `console.log`, um hook que barra secret e uma imagem Docker da aplicação rodando localmente. Os objetivos refletem essa entrega.

Aplicar

Configurar Husky, lint-staged e scanner de secrets em projeto Node, com os quatro hooks do Git na ponta do dedo.

Construir

Escrever um Dockerfile que empacote uma aplicação Node, com multi-stage build e `.dockerignore` bem pensados.

Inspecionar

Rodar um scanner de CVE na imagem gerada e ler o resultado sem se assustar com o número total.

Conectar

Enxergar como hook local, build reprodutível e scan de imagem viram política de merge na Aula 06.

Da revisão ao commit, ponte com a Aula 04

inteli.

Na Aula 04 a gente olhou qualidade como processo: revisão humana, análise estática com Sonar, IA como terceiro revisor. Todo esse controle aparece depois que o código chega no repositório remoto. A pergunta desta aula é uma só: o que deveria ter acontecido antes?

Aula 04, depois do push

- MR aberta, revisor olhando o diff
- Sonar rodando na pipeline
- IA comentando padrão sintático
- Decisão de merge no final

Aula 05, antes do push

- Hook bloqueia commit inválido
- Lint e teste só nos arquivos tocados
- Scanner de secret antes do histórico
- Imagem Docker builda antes do CI

Por que isso muda o jogo

Defeito pego no laptop custa segundos. O mesmo defeito pego na pipeline custa minutos, porque o dev já trocou de contexto. Em produção custa o dia de alguém. A distribuição de onde a barreira fica define quanto a engenharia gasta em retrabalho.

Três barreiras antes do push

inteli.

A aula de hoje está organizada em três camadas de validação local. Cada uma pega um tipo de problema diferente. Quem pula qualquer uma delas está pagando pedágio mais caro na pipeline.

1, Hook de commit

Formatação, lint, teste rápido e mensagem padronizada. Husky e lint-staged. Roda em segundos, barra o commit se falhar.

2, Scanner de secrets

TruffleHog ou Gitleaks como hook. Varre o diff, barra se detectar credencial. Última chance antes do histórico do Git.

3, Build em container

Se a imagem Docker não builda no laptop, não vai buildar no CI. Dockerfile reprodutível elimina "na minha máquina funciona".

Três barreiras, um mesmo princípio

Empurrar feedback para o mais cedo possível. O desenvolvedor fica no mesmo contexto, a correção é barata, o ciclo se fecha rápido. Shift-left não é buzzword, é matemática de custo por tempo de detecção.

BLOCO 1, 30 MIN

Git hooks e secrets, a primeira barreira

Quatro hooks, Husky, lint-staged, scanner de secrets. Vocês saem da sala com isso rodando no repo.

Os quatro hooks que importam

inteli.

Git tem mais de dez hooks. Na prática do dia a dia, quatro resolvem 95% dos casos. Dois rodam no cliente (laptop), dois no servidor. Todos são scripts shell comuns na pasta `.git/hooks`. Husky só empacota esses scripts em uma configuração versionável.

CLIENTE

`pre-commit`

Roda antes de o commit ser criado. Bloqueia se o lint quebrar ou o teste falhar. É o 80% do uso.

CLIENTE

`commit-msg`

Valida a mensagem do commit. Lugar natural do commitlint, forçando Conventional Commits.

CLIENTE

`pre-push`

Antes de enviar ao remoto. Bom para rodar a suite completa, teste de integração, coisas mais demoradas.

SERVIDOR

`pre-receive`

Roda no servidor Git antes de aceitar o push. Aqui mora política: proteção de branch, regras organizacionais.

Hook é script shell, nada de mágica

Abre `.git/hooks/pre-commit.sample` em qualquer repositório clonado. É shell. Husky versiona esses scripts na pasta `.husky/` do projeto, então toda a equipe herda o mesmo comportamento ao clonar. Sem Husky, cada dev tem o próprio hook, o que já é ter nenhum.

O trio que a Sprint usa

inteli.

No repositório da Sprint, três pacotes trabalham juntos. Cada um resolve uma fatia do problema e nenhum substitui o outro. O arquivo `.husky/pre-commit` chama o `npx lint-staged`, que por sua vez consulta o `package.json` para saber o que rodar em cada tipo de arquivo.

Husky

Instala os hooks na pasta `.husky/` e garante que o Git use eles. Configuração mora no repo, não no laptop.

lint-staged

Roda comando só nos arquivos do stage. Editou dois arquivos em cem, ele rodou lint em dois. Feedback em segundos.

commitlint

Valida a mensagem contra a regra Conventional Commits. `feat:`, `fix:`, `chore:`. Commit vago não passa.

```
// package.json, trecho do lint-staged
"lint-staged": {
  "*.{js,jsx,ts,tsx}": ["eslint --fix", "prettier --write"],
  "*.{md,json}": ["prettier --write"]
}
```

Scanner de secrets, TruffleHog e Gitleaks

inteli.

Um `AWS_SECRET_ACCESS_KEY` vazado no Git é caro de remover depois. BFG, filter-branch, rotacionar a credencial no provedor, tudo isso dá trabalho e assusta auditoria. Melhor barrar antes. Dois scanners dominam o mercado hoje, com filosofias diferentes.

TruffleHog

- Detecção por **entropia** e por regex
- Verifica validade das credenciais no provedor
- Varredura no histórico inteiro do repo
- Go, standalone, roda em pipeline também

Gitleaks

- Detecção por **regex** com ruleset customizável
- Baseline para ignorar falsos positivos conhecidos
- Integra nativamente como hook pre-commit
- Go, config em TOML, CI amigável

O que a Sprint usa

O repositório do módulo vem com TruffleHog configurado como hook pre-commit via Husky. Varre diff, falha se detectar padrão de credencial. No Exercício A vocês ativam, tentam commitar um secret de mentira e veem a barreira cair em ação.

HANDS-ON A, EM DUPLA, 15 MIN

Ativar hook e tentar vazar um secret

1. Clone `github.com/josercf/inteli-2026-1B-T13-ES10-S3`, rode `npm install`. O postinstall já ativa o Husky.
2. Abra qualquer arquivo e adicione a linha `AWS_SECRET_ACCESS_KEY="AKIA1234567890EXAMPLE"`.
3. Tente `git add .` seguido de `git commit -m "wip"`. Observe o que acontece na terminal.
4. Conserte: remova a linha do secret, ajuste a mensagem para `feat: testar hook`, commit de novo.
5. Confirme que o segundo commit passou e anote uma frase sobre o que o hook imprimiu na tela.

Se o commit vago passar, seu Husky não está ativo. Rode `npx husky init`, verifique a pasta `.husky/`, tente de novo.

Debrief A, o que quase todo mundo quebra

inteli.

Toda turma tropeça nos mesmos três pontos quando ativa Husky pela primeira vez. Mapeio aqui para vocês não gastarem tempo no óbvio e pra gente discutir juntos em um minuto.

1, Hook não roda

Esqueceu `npm install`. Sem postinstall, o Husky não engata. Sintoma: commit passa sem mensagem de lint.

2, Lint falha sem diff

lint-staged rodou em arquivo que não está no stage. Confere se você fez `git add` só do arquivo que editou.

3, Secret passa por pouco

Usou string com entropia baixa, tipo `abc123`. TruffleHog ignora. Use o prefixo `AKIA` que ele reconhece.

A pergunta que fica

Se o hook roda só no laptop, um dev mal-intencionado desativa com `git commit --no-verify`. Por isso a barreira local é só uma das três. Na Aula 06 a gente volta para a mesma pergunta com scanner no servidor, que ninguém consegue pular.

BLOCO 2, 45 MIN

Containers, a unidade de build reprodutível

Virtualização, Docker, Dockerfile, best practices. Vocês saem com uma imagem do academic-tracker rodando no laptop.

Virtualização vs containers, a pilha importa

inteli.

Antes de Docker, reprodutibilidade significava máquina virtual. Rodar outra cópia do sistema operacional inteira, com kernel próprio, pra isolar aplicação. Containers fazem o mesmo isolamento compartilhando o kernel do host. Menos gordura, boot em segundo, densidade alta no mesmo hardware.

Máquina virtual

- App A
- App B
- Bins e libs (guest)
- Guest OS
- Hypervisor
- Host OS
- Hardware

Container

- App A
- App B
- Bins e libs (imagem)
- Docker Engine
- Host OS (kernel compartilhado)
- Hardware

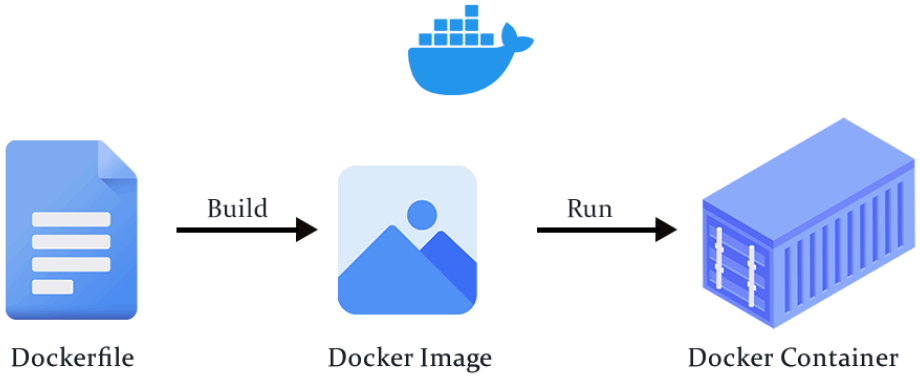
O isolamento que o kernel faz

Containers usam recursos do próprio kernel Linux: `namespaces` para separar visão de processo, rede, usuário, e `cgroups` para limitar CPU, memória, IO. Docker Engine orquestra essas duas primitivas. No Mac e Windows rodam dentro de uma VM leve, por isso Docker Desktop precisa de virtualização habilitada.

Dockerfile, imagem, container, o ciclo inteiro

inteli.

Três conceitos, três comandos, uma confusão bem comum. Dockerfile é a receita, arquivo de texto com instruções. Imagem é o artefato gerado pelo build, imutável, versionada por tag. Container é uma instância da imagem rodando, efêmero por natureza.



Dockerfile vira imagem no `docker build`*. Imagem vira container no* `docker run`*.*

Mnemônico útil

Dockerfile está para imagem como código-fonte está para binário. Imagem está para container como classe está para instância. Quando o aluno confunde, é porque esqueceu qual é a etapa imutável e qual é a viva.

Ler um Dockerfile linha a linha

inteli.

Esse é o Dockerfile que a gente vai usar no exercício. Dez linhas, cinco instruções distintas, multi-stage para reduzir tamanho da imagem final. Cada número corresponde a um comentário à direita.

```dockerfile
# 1
FROM node:20-alpine AS build
# 2
WORKDIR /app
# 3
COPY package*.json ./
# 4
RUN npm ci
# 5
COPY . .
RUN npm run build

# 6
FROM node:20-alpine
WORKDIR /app
# 7
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
# 8
EXPOSE 3000
# 9
CMD ["node", "dist/app.js"]
```

1. **FROM**, base image. Alpine tem 5 MB, Debian tem 100 MB.
2. **WORKDIR**, pasta padrão dentro do container.
3. **COPY** só o manifest primeiro, para cache.
4. **RUN npm ci** instala dependências de forma determinística.
5. **COPY . .** traz o código. Ordem importa para cache.
6. Segunda **FROM**, stage final, sem dev dependencies.
7. Pega só o `dist` do stage anterior.
8. **EXPOSE**, documenta a porta, não abre nada.
9. **CMD**, comando padrão quando o container sobe.

Dockerfile, três erros que toda primeira imagem comete

inteli.

Errar aqui é parte do processo, por isso os três pontos abaixo aparecem em praticamente toda revisão de primeiro Dockerfile. Quem já sabe disso pula uma viagem inteira de "por que minha imagem tem 1 GB".

1, Ordem das camadas

Copiar código antes do manifest invalida o cache de dependências a cada commit. `COPY package*.json` primeiro, `npm ci`, depois `COPY . .`. Economiza minutos por build.

2, Multi-stage build

Imagem de build tem compilador, node_modules de dev, cache. Imagem final não precisa de nada disso. Dois `FROM` no mesmo Dockerfile reduzem de 900 MB para 80 MB.

3, .dockerignore

Sem ele, o `COPY . .` empacota `node_modules`, `.git`, `coverage`, fotos. Imagem infla, secret em `.env` vai junto. Três linhas resolvem 90% do problema.

Teste rápido

Rode `docker history <imagem>`. Cada linha mostra o tamanho da camada e o comando que a criou. Se tem camada de 300 MB vindo de um `COPY . .`, o seu `.dockerignore` está pedindo socorro.

HANDS-ON B, EM DUPLA, 25 MIN

Construir imagem do academic-tracker

1. Na raiz do repo clonado, crie um arquivo `Dockerfile` e um `.dockerignore`. Use o modelo do slide anterior.
2. Rode `docker build -t academic-tracker:dev .` e espere o build completar. Anote o tamanho da imagem com `docker images`.
3. Suba um container: `docker run --rm -p 3000:3000 academic-tracker:dev`. Acesse `http://localhost:3000/health`.
4. Quebre de propósito: troque a ordem, ponha `COPY . .` antes de `npm ci`. Rebuilde e veja o cache falhar. Volte.
5. Rode `docker history academic-tracker:dev` e identifique a camada mais pesada. Em dupla, proponha uma otimização.

Se seu Docker Desktop não subir, forma dupla com quem tem. O tempo aqui é para experimentar, não para instalar. TI foi avisado, mas aula é aula.

Debrief B, o que a imagem gerada contém

inteli.

A pergunta que separa quem entendeu Dockerfile de quem só copiou é: o que está dentro da imagem que eu acabei de gerar? Quatro observações que quase sempre aparecem em debrief.

Tamanho fala

Imagem com 700 MB normalmente tem dev dependencies ou base Debian. Multi-stage e Alpine derrubam para 80 MB. Meça antes de otimizar.

Cache é ouro

Build que leva 3 min na primeira vez deve levar 10 segundos no segundo se só o código mudou. Se levar 3 min sempre, ordem das camadas está errada.

Imagem não é VM

Container não tem systemd, cron, ssh. Roda um processo principal e morre. Se você está pensando "como entro no container e mexo", é sinal de virar código.

Pergunta de bolso

Se amanhã eu deletar o seu laptop, a imagem que você gerou roda igual em outra máquina? Se a resposta for "depende", a reprodutibilidade ainda não chegou. Dockerfile bem feito tira o depende.

BLOCO 3, 20 MIN

Registry e vulnerabilidades, onde a imagem vai morar

Docker Hub, GHCR, ECR, Trivy. Imagem entra no pipeline, pipeline entra na Aula 06.

Registry, o GitHub das imagens

inteli.

Registry é servidor de artefato com namespace, autenticação e tag. `docker push` envia, `docker pull` traz de volta. A imagem que roda em produção passou por algum registry. Quatro são os mais comuns no mundo real, com tradeoffs diferentes.

Docker Hub

O mais conhecido, público por default. Ótimo para imagens oficiais e open source. Para privado, conta paga. Rate limit é real.

GHCR e GitLab Registry

Embutidos no próprio GitHub/GitLab. Permissão herdada do repo, pipeline publica direto. Escolha natural quando o repo já está lá.

AWS ECR, Azure ACR, GCP AR

Registry do cloud provider. Privado por padrão, integra com IAM, scanner embutido. Opção óbvia se o deploy vai para essa nuvem.

Sobre tag latest e imutabilidade

Tag `latest` não é imutável. Duas pessoas fazem `docker pull node:latest` em dias diferentes e podem receber imagens diferentes. Em produção isso é uma armadilha. Use tags semânticas com versão explícita, ex. `academic-tracker:1.4.2`, e considere digest SHA para garantir imutabilidade absoluta.

Toda imagem carrega CVEs

inteli.

A imagem da sua aplicação é feita de três superfícies: o código que você escreveu, as dependências do `package.json` e a base image. As duas primeiras a gente já escaneia com ferramentas como npm audit. A terceira é o ponto cego. `node:20-alpine` que você puxou na semana passada pode ter CVE novo que o de hoje já corrige.

O que é CVE

Common Vulnerability and Exposure. Identificador padrão para falha conhecida, tipo `CVE-2024-12345`. Base image herda CVE do sistema operacional, do OpenSSL, do glibc.

O que Trivy faz

Escaneia a imagem, casa com base de dados de CVE e lista por severidade. CLI aberta, roda em 30 segundos numa imagem pequena. Integra com pipeline.

Decisão, não lista

A saída de scanner sempre assusta. O trabalho é triar: o que é exploitable no meu contexto, o que é falso positivo, o que exige atualizar base image já.

Regra prática

Reconstrua sua base image toda semana, mesmo sem mudar nada. `docker pull node:20-alpine` e rebuild da imagem. Isso já elimina 80% dos CVEs que um scanner vai reclamar. O resto é triagem consciente.

Demo, Trivy na imagem que vocês acabaram de gerar

inteli.

No projetor, eu rodo `trivy image academic-tracker:dev` na imagem que a turma produziu no Exercício B. A saída típica num Alpine recente é a seguinte. Preste atenção em severidade, pacote afetado, versão fixada e vetor de ataque.

```
academic-tracker:dev (alpine 3.19)
==========================================
Total: 8 (UNKNOWN: 0, LOW: 3, MEDIUM: 2, HIGH: 2, CRITICAL: 1)

┌────────────────┬────────────────────┬──────────┬──────────────────┐
│ Library        │ Vulnerability      │ Severity │ Fixed Version    │
├────────────────┼────────────────────┼──────────┼──────────────────┤
│ libcrypto3     │ CVE-2024-5535      │ CRITICAL │ 3.1.4-r6         │
│ libssl3        │ CVE-2024-4741      │ HIGH     │ 3.1.4-r5         │
│ busybox        │ CVE-2023-42366     │ HIGH     │ 1.36.1-r15       │
│ ca-certs       │ CVE-2023-46121     │ MEDIUM   │ 20240226-r0      │
│ ...            │ ...                │ ...      │ ...              │
└────────────────┴────────────────────┴──────────┴──────────────────┘
```

Como ler isso na prática

CRITICAL e HIGH exigem ação. Atualizar base image costuma resolver os dois de uma vez. MEDIUM e LOW entram em fila de triagem. Severidade não conta contexto, então o humano ainda decide se um CVE medium em lib interna é blocker ou ruído.

Gancho para a Aula 06, isso tudo vira gate

inteli.

Até aqui a gente montou as barreiras no laptop. Elas funcionam pela disciplina do dev. Um `git commit --no-verify` pula tudo. Por isso a Aula 06 pega essas mesmas validações e move para um lugar que ninguém consegue pular, o pipeline remoto.

SAST no pipeline

Sonar, Semgrep, CodeQL. Mesmos apontamentos que a Aula 04 viu localmente, agora bloqueando merge por severidade configurada.

Secrets no runner

TruffleHog ou Gitleaks rodando em job da pipeline. Segundo tiro no mesmo alvo, caso algum dev tenha ignorado o hook local.

Image scanning

Trivy como step do GitHub Actions. Falha a build se CVE CRITICAL aparece. Política de merge passa a depender do resultado do scan.

O que muda na sua cabeça

Depois da Aula 06 você vai entender que "build verde" não é só teste passando. É código sem smell de segurança, sem secret, com imagem limpa de CVE. Qualidade vira estado observável, não promessa de revisor cansado.

FECHAMENTO, 10 MIN

Checagem, checklist, próxima aula

Duas perguntas rápidas, cinco linhas de checklist, um tema para a semana.

Checagem 1 de 2, hooks

inteli.

Você quer bloquear um commit que contém a palavra `console.log` em arquivo JavaScript. Qual hook é o natural para essa validação?

- `commit-msg`, porque valida texto
- `pre-commit`, porque roda antes do commit ser criado e tem acesso ao diff
- `pre-push`, porque é antes do remoto
- `pre-receive`, porque é onde política mora

Checagem 2 de 2, Dockerfile

inteli.

Você escreveu um Dockerfile começando com `FROM node:latest`. Sua colega de dupla sugere trocar por `FROM node:20.11.1-alpine`. Qual é o ganho concreto?

- A imagem com `latest` é mais lenta para buildar
- Alpine é mais seguro por si só, independente da versão
- Versão fixa garante build reprodutível. Alpine reduz superfície de ataque e tamanho
- Docker não aceita mais `latest` em produção

Checklist antes do push, para grudar no monitor

inteli.

Cinco linhas que a equipe madura roda antes de `git push` sem nem pensar. Quando vocês fecharem a sprint, essa lista deveria ser automática. Se alguma ficar no manual, vira fila de nit na revisão do parceiro.

1, Lint
ESLint e Prettier sem warning nos arquivos tocados.

2, Teste
Unit tests verdes no diff, integração se afeta rota.

3, Commit
Mensagem em Conventional, validada por commitlint.

4, Secret
TruffleHog ou Gitleaks não achou nada suspeito.

5, Imagem
`docker build` completa sem erro na sua máquina.

A regra do carrinho

Pipeline que quebra na número 5 quando 1, 2, 3, 4 passaram é pista de que o ambiente local divergiu do runner. Dockerfile bem feito torna esse caso raro. Pipeline que quebra em 1, 2 ou 3 é sinal de que o hook não rodou, ou o dev passou com `--no-verify`. Nos dois casos, a conversa é outra.

Auto-estudo para a Aula 06

inteli.

A Aula 06 leva tudo isso para o pipeline remoto e fecha a Sprint 02 com SAST e DAST. Quatro leituras curtas antes da próxima aula, nessa ordem.

1, Atlassian, 15 min

"What is Continuous Integration?". Revisa o vocabulário de CI sem assumir ferramenta. Base comum para a discussão de pipeline.

2, GitLab Docs, 30 min

"Static Application Security Testing (SAST)". Fica com o que é SAST, como o GitLab aplica e como ler o widget de achados na MR.

3, OWASP, 30 min

OWASP Top 10, edição 2021. Leitura rápida das dez categorias. Dê atenção a A01, A03 e A05, que dominam o ranking.

4, Semgrep, 20 min

"Getting started with Semgrep". Tour pelo CLI e pelo formato de regra. Ajuda a seguir o exercício hands-on da Aula 06.

Entrega desta aula, para o AdaLove

Um print do terminal mostrando o hook barrando o secret. Um print do `docker images` com a imagem do academic-tracker listada e seu tamanho. Uma linha dizendo qual foi o CVE mais sério que o Trivy apontou na sua imagem. Curto, objetivo, vira diagnóstico da sprint.

inteli.

Até a próxima

Aula 06, CI parte 2, SAST e DAST no pipeline, 13/05